

MARL-Based Trust Evaluation for Zero-Trust Low-Altitude UAV Networks

Anonymous Authors

Department of Computer Science and Engineering, Anonymous Institution

Correspondence: to be provided after peer-review anonymization is removed.

Abstract: Low-altitude UAV networks face significant security challenges due to rapidly changing topology, intermittent links, and limited onboard resources, making them vulnerable to spoofing, tampering, and collusive attacks. Existing trust evaluation schemes often suffer from degraded detection accuracy under high malicious-node ratios because they rely on static trust weighting or centralized coordination. To address these limitations, this paper proposes MTIM, a Multi-Agent Reinforcement Learning (MARL)-based trust evaluation mechanism for continuous zero-trust verification in low-altitude UAV networks under a bounded simulation threat model. MTIM integrates Summarization Agents, five verified trust branches, an Intelligent Decision Agent, distributed Thinker Agents, confidence-aware aggregation, and temporally decayed status updates. The executable artifact implements MTIM as an offline-trained parameterized MARL trust-policy surrogate and provides PyTorch training code, a trained checkpoint, learned risk-only baselines, raw per-seed logs, statistical scripts, plotting scripts, paper-level trust baseline reimplementations, and an ns-3 trace-driven validation path. In the 50-seed trust-stream evaluation, MTIM achieves 94.4% detection accuracy, 91.8% F-score, and 90.5% task success rate under a 30% malicious-node ratio. A strong MLP-Trust baseline reaches statistically indistinguishable detection performance (94.7% accuracy and 92.1% F-score), while MTIM improves task success by 2.2 percentage points through incentive coupling. A bundled 500 s ns-3-derived trace is also evaluated as a sanity check, but multi-seed ns-3 packet-level validation and real-flight deployment remain outside the present evidence scope.

Keywords: UAV networks; trust management; multi-agent reinforcement learning; zero trust architecture; FANET security

1 Introduction

Low-altitude UAVs are becoming critical infrastructure for logistics, inspection, sensing, and emergency response, and their flexibility and rapid deployment make them a cornerstone of the emerging low-altitude economy [1]. However, the openness and mobility that enable these applications also greatly enlarge the attack surface of UAV networks. In flying ad hoc networks (FANETs), three fundamental characteristics complicate security protection: highly dynamic topology, intermittent connectivity, and severely constrained onboard resources. These characteristics make malicious node infiltration, data tampering, spoofing, and collusion particularly difficult to detect and isolate [2].

Trust management has emerged as a core security primitive for UAV networks, enabling continuous assessment of node trustworthiness to detect malicious activities and mitigate attack impacts. However, existing trust management schemes still face three persistent limitations:

First, many existing methods rely on fixed rules or centralized coordination, which weakens resilience in infrastructure-scarce missions common in low-altitude UAV operations. Second, recent approaches such as GDM-DTM, FUBA, and GALTrust either depend on manually designed trust weighting, lack adaptive mechanisms for dynamic environments, or are optimized for static IoT settings rather than highly dynamic UAV swarms. They often fail to effectively detect coordinated attacks that simultaneously affect communication quality, task execution, and energy behavior [8, 10, 23]. Third, most methods lack adaptive mechanisms that can dynamically adjust trust strategies in response to changing attack patterns, leading to significant accuracy degradation when malicious nodes account for a high proportion of the swarm.

Zero trust offers a principled security perspective for this setting, built on the core principle of “never trust, always verify”, which requires continuous context-aware verification rather than static perimeter-based trust [15]. However, applying zero trust to UAV swarms requires more than just authentication. It

also requires online trust estimation, uncertainty management, and adaptive response mechanisms that can operate effectively under mobility, partial observability, and limited resources. Meanwhile, multi-agent reinforcement learning (MARL) has shown strong potential for distributed control and optimization in UAV systems [16], but it has rarely been leveraged to jointly optimize trust evaluation and incentive design, where trust assessment itself becomes the optimization objective to shape long-term trustworthy behavior.

Problem statement. This paper addresses the trust-layer decision problem in fixed-identity low-altitude UAV swarms: given noisy multi-factor trust evidence under dynamic topology and bounded behavioral attacks, how can a learned trust policy continuously classify node risk and convert trust decisions into mission-level incentives without relying on centralized infrastructure or online labels? This scope deliberately focuses on continuous trust verification and task-level response, rather than complete identity management, Sybil-resistant admission control, or real-flight deployment.

This paper proposes MTIM, a MARL-based trust evaluation mechanism for continuous zero-trust verification in low-altitude UAV networks. As shown in Figure 1, MTIM integrates Summarization Agents, multi-dimensional trust branches, an Intelligent Decision Agent, and distributed Thinker Agents into a unified closed-loop framework: trust assessment produces node credibility scores, the incentive mechanism rewards high-trust nodes and penalizes low-trust nodes, and an offline-trained MARL-style PyTorch policy maps temporal trust evidence to risk, trust-factor weights, and incentive actions under the bounded adversary model defined in Section 3.2. The main contributions of this paper are as follows:

- **A unified MARL-based framework for trust evaluation and incentive design:** Existing methods treat trust evaluation as a fixed scoring task rather than a closed-loop process that shapes node behavior. We formulate trust evaluation and incentive allocation as a joint MARL-style policy-learning problem so that the learned policy can improve mission continuity under fixed-identity UAV swarms.
- **Multi-factor trust evaluation with cross-validation:** Single-factor or manually weighted models fail to detect coordinated attacks affecting multiple dimensions simultaneously. We design a trust evaluation mechanism with five complementary dimensions—Behavioral Trust, Communication Trust, Energy Trust, Task Execution Trust, and Historical Trust—and a cross-factor consistency checking mechanism. In the current artifact, this design improves detection by 1.5 pp in accuracy relative to disabling directional cross-factor features, while stronger cross-dimension deception tests remain necessary.
- **Distributed MARL-style trust-policy execution:** Static trust weighting cannot adapt to dynamically changing network conditions. We develop a distributed Thinker-Agent architecture where each agent supervises a subset of UAV nodes and executes an offline-trained MARL-style policy that outputs malicious risk, trust-factor weights, and incentive actions from local observations.
- **Intelligent decision aggregation with uncertainty quantification and dynamic evolution:** Local multi-agent assessments inevitably involve bias and uncertainty. We introduce an Intelligent Decision Agent that fuses verified branch outputs and Thinker-Agent estimates into globally consistent trust scores with explicit uncertainty quantification. Combined with temporal decay, this approach keeps trust values aligned with both historical behavior and recent observations.

The remainder of this paper is organized as follows. Section 2 reviews related work on trust management in UAV networks and MARL for network security. Section 3 presents the system model, including the multi-factor trust evaluation mechanism, the Dec-POMDP modeling perspective, the MARL-style offline policy objective, and the state update dynamics. Section 4 describes the experimental setup and presents the performance evaluation results. Section 5 concludes the paper and outlines future research directions.

2 Related Work

Trust evaluation for UAV networks has evolved from static reputation mechanisms to evidence-fusion, blockchain-enabled, and learning-driven designs. However, the combination of zero-trust security principles, distributed trust learning, and uncertainty-aware aggregation remains underdeveloped in low-altitude UAV networks. This section reviews prior work from three perspectives: trust management in

UAV/FANET systems, zero-trust and continuous trust assessment, and MARL for UAV networking and security.

2.1 Trust Management in UAV and FANET Systems

Recent surveys show that trust management has become a core security primitive in FANETs, where dynamic topology, intermittent links, and resource limits make purely identity-based protection insufficient [1–5]. Existing schemes fall into three categories: behavior-centric trust analytics, distributed or blockchain-supported trust, and learning-enhanced trust evaluation.

Behavior-centric trust evaluation widely uses fuzzy and evidence-fusion methods to combine heterogeneous observations. Benfriha et al. proposed FUBA, which uses fuzzy logic to integrate indicators such as RSSI, packet delivery ratio, delay, battery level, and humidity to identify insider threats in FANETs [8]. Li et al. developed an adaptive multi-granularity trust management scheme for UAV visual sensor security under adversarial attacks, where coarse- and fine-grained evidence is fused to improve robustness against known and unknown threats [9]. Akram et al. introduced GALTrust, which combines generative adversarial learning with type-2 fuzzy logic to improve adaptability to emerging malicious behaviors in spatial crowdsourcing drone services [10]. More recent federated-security studies also move trust assessment toward collaborative anomaly detection, including FL-based intrusion detection for FANETs, multimodal swarm anomaly detection, and communication-efficient FL for resource-constrained UAV-IoT systems [11–13]. These studies significantly enrich trust evidence modeling, but most remain tightly coupled to specific sensing, communication, or crowdsourcing scenarios, and generally rely on hand-crafted fusion pipelines or fixed trust update logic.

A second line of work improves trust persistence and auditability through distributed storage or hierarchical trust exchange. Ge et al. developed a provenance-aware distributed trust model for resilient UAV networks, highlighting the value of peer-to-peer evidence tracing in hostile environments [7]. Zuo et al. proposed a hierarchical blockchain-based trust measurement method for drone clusters to address trust inheritance, cross-cluster re-evaluation, and historical trust verification under dynamic node entry and exit [6]. Akram et al. proposed TMIoT, which integrates digital twins, blockchain, and deep-learning-based intrusion detection to secure Open RAN-based Internet of Drone Things applications [17]. Closely related secure-FL frameworks have further explored malicious-node detection and trusted collaboration through blockchain-supported service provisioning, privacy-preserving IoT security, and hierarchical federated aggregation [18–21]. Related work on secure quantum federated learning in UAV-assisted wireless networks further indicates that trust-enhanced game incentives can improve participation quality and strengthen learning robustness under strategic behavior [22]. More recently, Hu et al. proposed GDM-DTM for malicious-node detection in low-altitude UAV networks by combining group decision making with dynamic trust management [23]. Although these schemes improve distributed credibility and malicious-node detection, most depend on manually designed trust weighting and update rules, limiting their adaptability to coordinated multi-factor attacks and rapidly changing mission conditions.

2.2 Zero-Trust and Continuous Trust Assessment

Zero trust shifts security from perimeter-based assumptions to continuous, resource-centric verification. The NIST zero-trust architecture defines this paradigm through the principle of “never trust, always verify” and replaces implicit perimeter-based trust with continuous, context-aware verification [15]. Building on this idea, Wang et al. proposed a zero-trust-driven collaborative intrusion detection framework for IoT that uses continuous trust assessment to support distributed security decisions [24]. In the UAV and 6G context, Al Ridhawi and Aloqaily integrated zero trust, digital twins, federated learning, and blockchain into a UAV-enabled 6G framework, highlighting the broader relevance of decentralized trust enforcement in highly dynamic aerial networks [25]. In the UAV authentication domain, Wang et al. introduced a context-aware trust evaluation model for malicious UAV detection during authentication, showing that trust can also be injected into the authentication stage rather than being treated purely as a post hoc reputation signal [14].

Zero trust is particularly relevant to low-altitude UAV swarms: node roles may change during flight, links may appear and disappear rapidly, and local observations may be incomplete or noisy. Trust decisions cannot be based solely on prior membership or static credentials, but must be continuously updated from current observations, device status, communication behavior, and mission context. Therefore, trust

decisions should be dynamic, uncertainty-aware, and resource-conscious rather than static or binary.

These studies move trust evaluation closer to continuous verification. Nevertheless, the current zero-trust literature still focuses mainly on authentication, access control, or intrusion detection. It does not fully address how trust should evolve online in low-altitude UAV swarms when heterogeneous evidence from behavior, communication quality, task execution, energy state, and historical interactions must be fused under strict resource constraints. In particular, uncertainty-aware aggregation and decentralized incentive adaptation remain underexplored in zero-trust UAV settings.

2.3 MARL for UAV Networking and Security

MARL has recently become a prominent framework for decentralized decision making in UAV systems. A recent survey by Ekechi et al. shows that MARL has mainly been used for UAV control, formation, coverage, coordination, and cooperative optimization, rather than for trust management itself [16]. Representative examples include the MADDPG-based multi-UAV trajectory planning framework of Wang et al. for UAV-assisted mobile edge computing [26], the semi-distributed multi-agent federated reinforcement learning approach of Nie et al. for resource management in UAV-aided MEC systems [27], and the joint optimization of trajectory control, task offloading, and resource allocation in air-ground integrated networks by Alam and Moh [28]. Closely related learning-and-optimization studies in UAV-enabled federated settings have examined asynchronous scheduling, contract-based participant selection, hierarchical personalized learning, air-ground FL coordination, and energy-aware trajectory and resource management [29–34]. These works confirm that distributed learning is well suited for optimization under partial observability and dynamic coupling among UAVs. More recently, Cao et al. explored LLM-based multi-agent situation awareness for zero-trust space-air-ground integrated networks [35]. Their results further suggest that model selection in security-oriented multi-agent systems should balance capability and deployment cost; for specialized subtasks, smaller task-adapted models can be more practical than larger general-purpose models.

However, existing MARL applications in UAV networking remain largely performance driven, with objectives such as delay reduction, energy efficiency, coverage, or offloading efficiency. The reward design in these studies is rarely aligned with trustworthiness, collusion resistance, or long-horizon security incentives. MARL has been used to optimize how UAVs move, communicate, or allocate resources, but not how they collaboratively evaluate trust, calibrate trust-factor importance, and propagate trustworthy decisions under zero-trust assumptions.

Overall, prior work has made strong progress in trust modeling, distributed storage, and UAV-oriented MARL, but three key challenges remain largely unaddressed:

- (1) **Accurate trust assessment under high malicious-node ratios:** Existing methods often suffer from significant accuracy degradation when malicious nodes account for a large proportion of the swarm, leading to poor mission continuity.
- (2) **Adaptability to coordinated multi-factor attacks:** Most existing schemes rely on single-factor metrics or manually designed trust weighting, which cannot effectively detect sophisticated coordinated attacks that simultaneously affect multiple dimensions of node behavior.
- (3) **Efficient continuous verification:** Blockchain-based approaches introduce coordination, storage, and consensus overhead that is difficult for resource-limited UAVs, while non-blockchain methods struggle to adapt trust updates to rapid topology and mission changes.

MTIM directly addresses these three challenges by combining multi-factor trust evaluation, MARL-style trust-policy execution, hierarchical Bayesian uncertainty-aware aggregation, and temporal trust decay into a unified continuous-verification framework for low-altitude UAV networks.

3 System Model

We consider a low-altitude UAV swarm executing a cooperative mission (e.g., area surveillance, delivery, or search-and-rescue) in an open, infrastructure-scarce environment. The network is subject to three intrinsic constraints—dynamic topology, intermittent connectivity, and limited onboard resources—that simultaneously degrade conventional perimeter-based defenses and amplify the impact of insider threats. When malicious nodes constitute a non-negligible fraction of the swarm, sparse observations make accurate trust inference a problem of *reasoning under uncertainty* rather than deterministic rule-checking.

This section formalizes the network model (Section 3.1), the threat model (Section 3.2), the multi-factor trust evaluation mechanism (Section 3.3), the Dec-POMDP formulation and MARL-style architecture (Section 3.4), and the trust state dynamics (Section 3.4.4).

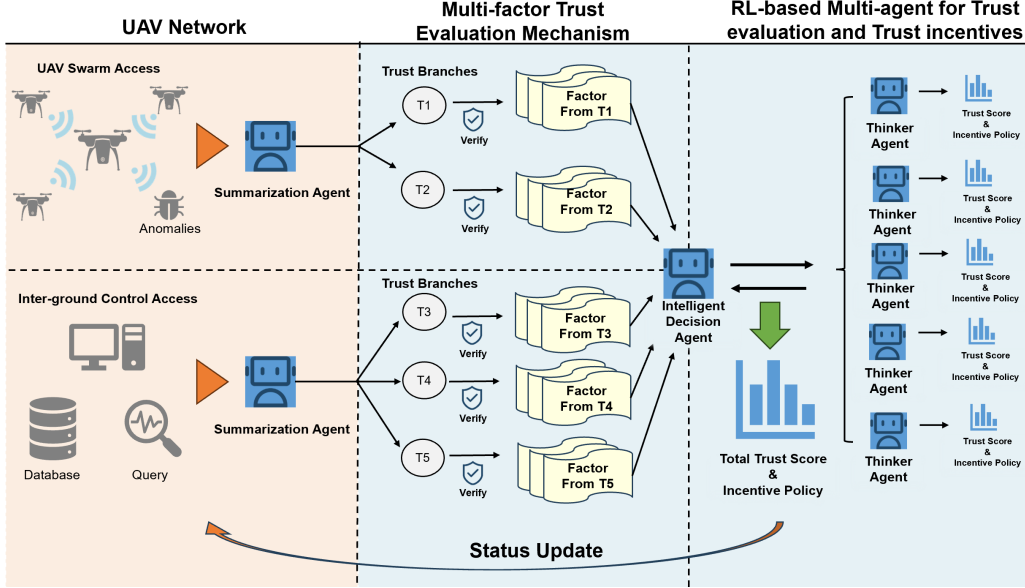


Figure 1: Overview of the MTIM framework. Summarization Agents collect UAV-network and ground-control observations, verified trust branches T1–T5 extract multi-factor evidence, the Intelligent Decision Agent forms the total trust score and incentive policy, and parallel Thinker Agents learn trust-score and incentive-policy decisions that close the status-update loop.

3.1 UAV Network Model

We model the low-altitude UAV network as a dynamic ad hoc network of N heterogeneous UAV nodes, $\mathcal{U} = \{U_1, U_2, \dots, U_N\}$. Each node U_i is characterized by a tuple $\langle \mathbf{p}_i(t), v_i(t), E_i(t), C_i \rangle$, where $\mathbf{p}_i(t) \in \mathbb{R}^2$ is the 2D position at time t , $v_i(t) \in [v_{\min}, v_{\max}]$ is the velocity, $E_i(t) \in [0, E_i^{\max}]$ is the residual energy, and C_i denotes the computational capacity (CPU cycles/s). Nodes may enter or leave the swarm during the mission according to operational requirements.

The communication topology is a time-varying undirected graph $G(t) = (\mathcal{U}, \mathcal{E}(t))$, where an edge $(i, j) \in \mathcal{E}(t)$ exists iff the Euclidean distance satisfies $d_{ij}(t) = \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\|_2 \leq R_{\text{comm}}$, with $R_{\text{comm}} = 300$ m (Table 2). Each link (i, j) has a time-varying quality characterized by packet delivery ratio $\text{PDR}_{ij}(t)$, received signal strength $\text{RSSI}_{ij}(t)$, and one-way latency $L_{ij}(t)$. The network is *infrastructure-scarce*: no fixed base stations or ground controllers are assumed to be continuously reachable; all trust computation and decision-making must execute on-board or via peer-to-peer exchange.

3.2 Threat Model

We adopt a white-box adversary model that is standard in the trust management literature [8, 23] but extend it to capture coordinated multi-dimensional manipulation.

Adversary capabilities. A subset of nodes $\mathcal{M} \subset \mathcal{U}$ ($|\mathcal{M}|/N \in [0.05, 0.50]$ in our experiments) is controlled by an adversary. Each malicious node $U_m \in \mathcal{M}$ can independently manipulate any subset of the five trust dimensions by:

- **T1 (Behavioral):** deviating from planned flight paths, skipping scheduled heartbeats, or responding inconsistently to coordination messages;
- **T2 (Communication):** performing selective forwarding (dropping a fraction $p_{\text{drop}} \in [0.3, 1.0]$ of packets), injecting spurious traffic, or manipulating transmission power to create asymmetric links;

- **T3 (Energy)**: exhibiting anomalous consumption patterns (e.g., sudden depletion through jamming, or unusually low consumption by remaining idle during active mission phases);
- **T4 (Task)**: returning fabricated or degraded task results (e.g., falsified sensor readings, partial payload delivery);
- **T5 (Historical)**: this dimension is not directly manipulable by the adversary—it is an accumulated reputation signal maintained by MTIM—but its update is indirectly affected when the adversary manipulates T1–T4.

Attack strategies. We consider three canonical attack patterns that cover the spectrum from naive to evasive adversaries:

- (i) **Constant attack**: the malicious node exhibits adversarial behavior from $t = 0$ and maintains it throughout the mission. This models unsophisticated attackers and serves as a lower bound on detection difficulty.
- (ii) **Dormant attack**: the malicious node behaves honestly during an initial warm-up period ($t \leq T_{\text{warm}}$), accumulating trust, then switches to adversarial behavior. This models adversaries that attempt to evade initial screening.
- (iii) **On-off attack**: the malicious node toggles between honest and adversarial behavior with period T_{toggle} , attempting to keep its trust score near the classification boundary where decisions are most uncertain.

In mixed-attack experiments, each pattern accounts for one-third of the malicious population. The main evaluation focuses on direct behavioral manipulation in T1–T4. To clarify the boundary of the zero-trust claim, we also include a limited collusion stress test in the sensitivity analysis: a subset of malicious nodes submits inflated peer recommendations while continuing one of the three behavioral attack patterns. Full Sybil resistance, where a physical adversary creates many unlinkable identities, requires identity-binding and admission-control mechanisms outside the scope of MTIM and is treated as a deployment prerequisite rather than a solved problem.

Adversary knowledge. The adversary is assumed to know the set of trust dimensions used by MTIM but *not* the frozen policy parameters, per-node policy outputs $\omega_m(t)$, or the Intelligent Decision Agent confidence weights $\phi_{jk}(t)$. This models a capable but not omniscient attacker. The adversary may submit biased peer observations in the collusion stress test, but cannot create Sybil identities, tamper with the Intelligent Decision Agent, or poison the offline training data.

Design implications. Under this threat model, a robust trust mechanism must satisfy four requirements: (R1) no single trust dimension can dominate the aggregate score, otherwise an adversary focusing on that dimension evades detection; (R2) trust weights must depend on observed context, because the adversary’s choice of which dimensions to attack is unknown a priori and may change over time; (R3) temporal consistency must be enforced, because dormant and on-off attackers exploit stale trust scores; and (R4) peer evidence must be down-weighted when an agent’s historical estimates become unreliable. MTIM’s cross-factor consistency check, policy-generated trust weighting, temporal decay, and Intelligent Decision Agent confidence weighting directly address R1–R4, respectively. The resulting protection is a continuous verification mechanism, not a complete replacement for cryptographic identity management or Sybil-resistant admission control.

3.3 Multi-factor Trust Evaluation Mechanism

MTIM assesses UAV credibility through five complementary trust dimensions, each capturing a distinct behavioral modality. The design principle is that a malicious node may evade detection in any single dimension by mimicking honest behavior, but sustaining consistent deception across five independent evidence channels is exponentially harder. We first define each dimension, then present the aggregation and cross-validation mechanisms.

3.3.1 Trust Factor Decomposition

To comprehensively capture different aspects of node behavior and detect sophisticated attacks that may only manifest in specific dimensions, we decompose trust evaluation into five complementary trust dimensions. This multi-factor design is motivated by the observation that a malicious node may behave normally in one dimension to evade detection while exhibiting abnormal patterns in another; by covering multiple dimensions, we increase the likelihood of detecting such coordinated attacks. Although including more factors introduces some computational overhead, each dimension captures a fundamentally different source of evidence, and the distributed Thinker-Agent architecture keeps execution costs bounded.

As illustrated in Figure 1, the trust evaluation is decomposed into multiple trust branches, each corresponding to a distinct trust factor. Let the overall trust score of UAV node i at time t be denoted as $T_i(t) \in [0, 10]$, which is a function of five fundamental trust dimensions (the formal aggregation mechanism is presented in Section 3.3.2), where each trust branch $T_i^{(m)}(t) \in [0, 1]$ is defined as follows:

- **T1: Behavioral Trust** $T_i^{(1)}(t)$. This factor evaluates operational consistency, including adherence to flight paths, communication regularity, and response reliability:

$$T_i^{(1)}(t) = \frac{1}{N_b} \sum_{j=1}^{N_b} \mathbb{I}(a_{i,j}(t) = a_{i,j}^{expected}(t)) \quad (1)$$

where $\mathbb{I}(\cdot)$ is the indicator function, $a_{i,j}(t)$ represents the actual action in behavior category j , and $a_{i,j}^{expected}(t)$ is the expected behavior. This yields $T_i^{(1)}(t) \in [0, 1]$.

- **T2: Communication Trust** $T_i^{(2)}(t)$. This factor assesses link quality and stability through packet delivery ratio, signal strength, and latency:

$$T_i^{(2)}(t) = w_p \cdot PDR_i(t) + w_s \cdot \frac{RSSI_i(t)}{RSSI_{max}} + w_l \cdot \left(1 - \frac{L_i(t)}{L_{max}}\right) \quad (2)$$

where $PDR_i(t)$, $RSSI_i(t)/RSSI_{max}$, and $1 - L_i(t)/L_{max}$ are each normalized to $[0, 1]$, and $w_p = 0.4, w_s = 0.3, w_l = 0.3$ are weighting coefficients with $w_p + w_s + w_l = 1$. Thus $T_i^{(2)}(t) \in [0, 1]$.

- **T3: Energy Trust** $T_i^{(3)}(t)$. This factor monitors energy-consumption patterns and remaining battery life to detect anomalies indicative of malfunction or compromise:

$$T_i^{(3)}(t) = \frac{E_{remaining}(t)}{E_{initial}} \cdot \exp(-\alpha \cdot |C_i(t) - \bar{C}(t)|) \quad (3)$$

where $\frac{E_{remaining}(t)}{E_{initial}} \in [0, 1]$ and the exponential term with $\alpha = 0.5$ is also bounded in $[0, 1]$, yielding $T_i^{(3)}(t) \in [0, 1]$.

- **T4: Task Execution Trust** $T_i^{(4)}(t)$. This factor measures the success rate and quality of task completion, such as sensing, data relaying, or payload delivery:

$$T_i^{(4)}(t) = \frac{1}{N_t} \sum_{k=1}^{N_t} \left(\frac{Q_{i,k}(t)}{Q_k^{max}} \cdot \mathbb{I}(\text{task_success}) \right) \quad (4)$$

where each term is normalized to $[0, 1]$, so $T_i^{(4)}(t) \in [0, 1]$.

- **T5: Historical Trust** $T_i^{(5)}(t)$. This factor incorporates long-term reputation scores and past behavior records to preserve temporal continuity in trust evaluation:

$$T_i^{(5)}(t) = \beta \cdot T_i^{(5)}(t-1) + (1 - \beta) \cdot T_i^{inst}(t) \quad (5)$$

where both the historical and instantaneous components are in $[0, 1]$, and the smoothing factor $\beta = 0.7$. Thus $T_i^{(5)}(t) \in [0, 1]$.

3.3.2 Multi-factor Trust Aggregation

Each trust branch independently computes a sub-score $T_i^{(m)}(t) \in [0, 1]$ based on real-time data streams and historical logs. These sub-scores are then aggregated into a global trust score $T_i(t) \in [0, 10]$ through a weighted fusion mechanism:

$$T_i(t) = 10 \cdot \sum_{m=1}^5 \omega_m(t) \cdot T_i^{(m)}(t) \quad (6)$$

where $\omega_m(t)$ are dynamic weighting coefficients satisfying $\sum_{m=1}^5 \omega_m(t) = 1$, and the factor of 10 maps the aggregated score to the $[0, 10]$ range consistent with the classification thresholds $\theta_{\text{high}} = 7.5$ and $\theta_{\text{low}} = 3.0$. These weights are produced by the MARL-style trust policy introduced in the next section, allowing the system to emphasize more reliable trust factors under varying operational conditions.

3.3.3 Cross-Validation Mechanism

The multi-factor design ensures that no single point of failure or manipulation can compromise the overall trust decision. To enhance robustness, a **cross-validation mechanism** is implemented between trust branches. For any pair of trust factors m and n , the consistency metric is defined as:

$$\Gamma_{m,n}(t) = \left| T_i^{(m)}(t) - T_i^{(n)}(t) \right| \quad (7)$$

If $\Gamma_{m,n}(t) > \gamma_{\text{th}}$, where the cross-validation threshold $\gamma_{\text{th}} = 0.3$ (determined via sensitivity analysis over $\{0.1, 0.2, 0.3, 0.4, 0.5\}$ with 0.3 yielding the best F-score on a validation subset), a potential inconsistency is flagged, triggering deeper inspection. For instance, even if a UAV exhibits normal communication behavior ($T_i^{(2)}(t)$ high), but shows abnormal energy depletion ($T_i^{(3)}(t)$ low), the system can still flag it as potentially compromised. This layered verification aligns with the **zero-trust principle** of "never trust, always verify," where trust is continuously evaluated and never assumed.

Furthermore, the framework supports **anomaly detection** through statistical analysis of trust factor deviations:

$$\mathcal{A}_i(t) = \mathbb{I} \left(\sum_{m=1}^5 \left(\frac{T_i^{(m)}(t) - \mu_m(t)}{\sigma_m(t)} \right)^2 > \chi_{\alpha,5}^2 \right) \quad (8)$$

where $\mu_m(t)$ and $\sigma_m(t)$ are the mean and standard deviation of trust factor m across the swarm, $\chi_{\alpha,5}^2$ is the chi-squared threshold with 5 degrees of freedom at significance level $\alpha = 0.05$. This redundancy and diversity in trust evidence form the foundation for robust and adaptive trust management in UAV swarms, enabling the detection of sophisticated attacks that may manipulate multiple dimensions simultaneously.

3.4 MARL-Based Trust-Policy Learning and Incentive Model

Having defined the trust evidence channels, we now formulate the problem of *how to weight and act on this evidence* as a sequential decision process. MTIM employs M Thinker Agents, where each agent supervises a subset $\mathcal{N}_j \subset \mathcal{U}$ of UAV nodes, maps local trust evidence to risk-aware trust-factor weights, and issues incentive actions that shape task allocation. The closed-loop system is modeled as a Dec-POMDP and follows a MARL design pattern, but the executable artifact in this paper implements an *offline-trained parameterized MARL trust-policy surrogate*: the policy is trained from simulator-labeled trust streams and then frozen for all held-out evaluation seeds. We therefore avoid claiming online TD convergence or fully adaptive deployment-time reinforcement learning in the present artifact.

3.4.1 System Model and Problem Formulation

We consider a UAV swarm consisting of N UAV nodes. The trust management system employs M Thinker Agents, where each agent $j \in \{1, 2, \dots, M\}$ is responsible for a subset of UAV nodes. The overall system can be modeled as a Dec-POMDP (decentralized partially observable Markov decision process) [37]:

$$\mathcal{G} = \langle \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle \quad (9)$$

where $\mathcal{S}$ is the global state space, $\mathcal{O}$ is the joint observation space, $\mathcal{A}$ is the joint action space, $\mathcal{P}$ is the state transition probability, $\mathcal{R}$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor.

State Space: The state $s \in \mathcal{S}$ at time t is defined as the tuple of all UAV nodes' multi-factor trust scores:

$$s(t) = (\mathbf{T}_1(t), \mathbf{T}_2(t), \dots, \mathbf{T}_N(t)),$$

where $\mathbf{T}_k(t) = [T_k^{(1)}(t), T_k^{(2)}(t), T_k^{(3)}(t), T_k^{(4)}(t), T_k^{(5)}(t)]$ denotes the five-dimensional trust vector for node k , with each component in $[0, 1]$. The corresponding global trust score $\mathcal{T}_k(t) \in [0, 10]$ is computed by the Intelligent Decision Agent as described in Section 3.4.3.

Observation Space: Each Thinker Agent j observes a partial view of the global state, consisting only of the local trust sub-scores for the nodes in its supervised subset $\mathcal{N}_j$: $o_j(t) = (\mathbf{T}_k(t))_{k \in \mathcal{N}_j} \cup (c_{jk}(t))_{k \in \mathcal{N}_j}$, where $c_{jk}(t)$ is the confidence score assigned by agent j to its own assessment of node k .

Action Space: The action $a_j(t) \in \mathcal{A}_j$ for Thinker Agent j is a hybrid parameterized action consisting of (1) a continuous trust-weight vector $\omega_j(t) = [\omega_j^{(1)}(t), \dots, \omega_j^{(5)}(t)] \in \Delta^4$ on the probability simplex, and (2) a discrete incentive allocation vector $\pi_j(t) = [\pi_{jk}(t)]_{k \in \mathcal{N}_j}$ specifying the incentive level for each supervised node. Incentive levels are quantized into five bins $\{-2, -1, 0, 1, 2\}$, corresponding to strong penalty, weak penalty, neutral allocation, weak reward, and strong reward. If a sampled incentive vector violates the cycle budget $B_{\text{total}}(t)$, incentives are projected greedily by retaining actions with the largest expected utility gain per unit cost until the budget is satisfied.

Architecture: The implemented policy follows centralized offline training and decentralized execution, matching the CTDE motivation of MARL while using a supervised surrogate loss in the released artifact. During training, the simulator provides node labels only for the loss function; labels are never included in the deployed observations. During execution, each Thinker Agent acts only from local trust evidence and fixed learned parameters, ensuring that all reported decisions are label-free.

State Transition: The state transition $\mathcal{P}$ is defined implicitly through the trust update equations (Eqs. 23–24) in Section 3.4.4, which govern how trust scores evolve based on observations, incentives, and temporal decay.

3.4.2 Executable Parameterized MARL Trust-Policy Network

The artifact implements a compact PyTorch policy network whose inputs are per-node diagnostic features extracted from the post-warm-up trust streams. These features include recent weakness in T1, T2, T4, and T5; low-quantile T1/T4 evidence; low-score duration; T1/T4 amplitude and standard deviation; positive degradation slope; and directional cross-factor gaps. This feature mapping is implemented in `artifacts/scripts/marl_policy.py` and is used identically by MTIM and by the learned baselines introduced in Section 4.1.

For node k , let $\mathbf{x}_k \in \mathbb{R}^{13}$ denote the standardized diagnostic feature vector. A shared encoder maps $\mathbf{x}_k$ to a hidden representation h_k :

$$h_k = f_\theta(\mathbf{x}_k). \quad (10)$$

Four output heads are then computed:

$$\begin{aligned} p_k &= \sigma(g_r(h_k)), \\ \omega_k^{(m)} &= \frac{\exp(g_\omega^{(m)}(h_k))}{\sum_{\ell=1}^5 \exp(g_\omega^{(\ell)}(h_k))}, \\ \mathbf{q}_k^\pi &= g_\pi(h_k), \\ v_k &= g_v(h_k), \end{aligned} \quad (11)$$

where p_k is the malicious-risk probability, ω_k is the trust-factor weight vector, $\mathbf{q}_k^\pi$ contains the five incentive-bin scores, and v_k is an auxiliary value estimate. The local diagnostic trust score is

$$\hat{T}_{jk} = 10(1 - p_k), \quad (12)$$

and a node is flagged as malicious when $p_k > 0.60$. The selected incentive bin is

$$\pi_{jk}^{raw} = \arg \max_{\ell \in \{-2, -1, 0, 1, 2\}} q_{k,\ell}^\pi, \quad (13)$$

followed by the budget projection described above.

3.4.3 Offline MARL Surrogate Training Objective

The executable training objective is a MARL surrogate objective with supervised and auxiliary terms rather than an online temporal-difference update. Let $y_k \in \{0, 1\}$ be the simulator label used only during offline training. Let ω_k^* be a heuristic trust-weight target derived from the same diagnostic evidence, π_k^* an incentive-bin target derived from the label and risk hint, and v_k^* a bounded service-value target. The policy minimizes

$$\begin{aligned} \mathcal{L}(\theta) = & \text{BCE}_w(y_k, p_k) + 0.25 \|\omega_k - \omega_k^*\|_2^2 \\ & + 0.20 \text{CE}(\pi_k^*, \mathbf{q}_k^\pi) + 0.10 (v_k - v_k^*)^2. \end{aligned} \quad (14)$$

The primary risk term is weighted binary cross-entropy with a positive-class weight of 2.0. Training uses seeds 0–9 and a fixed decision threshold of 0.60. Evaluation seeds 1000–1049 are not used for training, threshold selection, or auxiliary-target tuning. This implementation makes the method-code correspondence explicit: MTIM’s reported policy is an offline-trained MARL-style trust-risk and incentive policy, not a deployment-time online reinforcement-learning implementation.

The Intelligent Decision Agent in Figure 1 serves as the decision node that combines verified trust-branch outputs and Thinker-Agent estimates to produce the total trust score and incentive policy. While this introduces a coordination point, MTIM adopts a **hierarchical** rather than fully decentralized architecture for three practical reasons: (1) confidence-aware fusion with explicit uncertainty quantification (Eq. 17) provides a principled reliability measure for downstream decisions; (2) the Intelligent Decision Agent does not require persistent storage or blockchain consensus—it operates statelessly on each update cycle, recomputing aggregation weights from agent error histories; and (3) the Intelligent Decision Agent can be replicated with Byzantine fault-tolerant consensus in safety-critical deployments, though we leave this extension to future work. The Intelligent Decision Agent uses the following confidence-weighted inference mechanism:

Total Trust Score Aggregation: The Intelligent Decision Agent computes the total trust score $\mathcal{T}_k(t)$ for each UAV node k by combining the individual trust scores from all Thinker Agents that have observed or interacted with node k :

$$\mathcal{T}_k(t) = \frac{\sum_{j=1}^M \phi_{jk}(t) \cdot \hat{T}_{jk}(t)}{\sum_{j=1}^M \phi_{jk}(t)} \quad (15)$$

where $\phi_{jk}(t) \in [0, 1]$ is the confidence weight assigned to Thinker Agent j ’s assessment of node k , computed based on the agent’s historical accuracy:

$$\phi_{jk}(t) = \frac{\exp(-\gamma \cdot \varepsilon_{jk}(t))}{\sum_{j'=1}^M \exp(-\gamma \cdot \varepsilon_{j'k}(t))} \quad (16)$$

with $\varepsilon_{jk}(t) = \frac{1}{t} \sum_{\tau=1}^t \left| \hat{T}_{jk}(\tau) - T_{jk}^{\text{obs}}(\tau) \right|$ representing the mean absolute error of agent j for node k over time, and $\gamma = 2.0$ a scaling parameter.

Uncertainty Quantification: The Intelligent Decision Agent also quantifies the uncertainty in global trust assessments using a Bayesian posterior variance:

$$\sigma_k^2(t) = \frac{\sum_{j=1}^M \phi_{jk}(t) \left(\hat{T}_{jk}(t) - \mathcal{T}_k(t) \right)^2}{\sum_{j=1}^M \phi_{jk}(t)} + \sigma_0^2 \quad (17)$$

where $\sigma_0^2 = 0.1$ represents the inherent measurement noise variance. This uncertainty metric is crucial for risk-aware decision-making in zero-trust environments.

Final Trust Classification: Based on the total trust score and uncertainty, the Intelligent Decision Agent classifies nodes into trust categories:

$$C_k(t) = \begin{cases} \text{Trusted} & \text{if } \mathcal{T}_k(t) \geq \theta_{\text{high}} \text{ and } \sigma_k(t) \leq \sigma_{\text{th}} \\ \text{Suspicious} & \text{if } \theta_{\text{low}} \leq \mathcal{T}_k(t) < \theta_{\text{high}} \text{ or } \sigma_k(t) > \sigma_{\text{th}} \\ \text{Malicious} & \text{if } \mathcal{T}_k(t) < \theta_{\text{low}} \text{ and } \sigma_k(t) \leq \sigma_{\text{th}} \end{cases} \quad (18)$$

where θ_{high} and θ_{low} are trust thresholds, and σ_{th} is the uncertainty threshold.

The Intelligent Decision Agent generates the incentive policy that coordinates the actions of individual Thinker Agents to promote network-wide trustworthiness. The incentive policy $\mathbf{\Pi}(t)$ is a mapping from global trust assessments to resource allocation decisions:

$$\mathbf{\Pi}(t) = \{\pi_{jk}(t) : j \in [1, M], k \in \mathcal{N}_j\}, \quad (19)$$

where $\pi_{jk}(t) \in \mathcal{A}_{\text{inc}} = \{-2, -1, 0, 1, 2\}$ is the incentive action for node k by agent j , encoding strong penalty (-2) through strong reward ($+2$). The policy is determined by solving a constrained optimization that maximizes expected network-wide utility:

$$\max_{\mathbf{\Pi}(t)} \mathbb{E} \left[\sum_{k=1}^N U_k(\mathcal{T}_k(t), \pi_{\cdot k}(t)) \right], \quad (20)$$

subject to the budget constraint $\sum_{j=1}^M \sum_{k \in \mathcal{N}_j} c(\pi_{jk}(t)) \leq B_{\text{total}}(t)$, where $c(\pi) = |\pi|$ quantifies the resource cost of each incentive action. The per-node utility function is:

$$U_k(\mathcal{T}_k(t), \pi_{\cdot k}(t)) = \mathcal{T}_k(t) \cdot (1 + \delta \cdot \mathbb{I}(\pi_{\cdot k}(t) > 0)) - \kappa \cdot \mathbb{I}(\mathcal{T}_k(t) < \theta_{\text{low}}), \quad (21)$$

with $\delta = 0.2$ and $\kappa = 2.0$. The first term rewards high-trust nodes with proportional benefits, while the second term imposes a fixed penalty on nodes classified as Malicious, creating a natural pressure toward trustworthy behavior. In practice, the optimization in Eq. 20 is solved greedily per update cycle due to the 50 s incentive interval constraint (Table 2).

3.4.4 State Update Mechanism for Trust Evolution

The state update mechanism governs how trust scores evolve over time as a function of node behavior, incentive actions, and network conditions. Trust evaluation uses Bayesian inference, and the overall trust dynamics constitute a Markov process because the trust score at time $t + 1$ depends only on the trust score at time t and the current observations and incentives, not on the full history. The state transition can be decomposed as:

$$\mathbf{T}(t + 1) = f(\mathbf{T}(t), \mathbf{A}^{\text{inc}}(t), \mathbf{A}^{\text{task}}(t), \mathbf{E}(t)) \quad (22)$$

where $\mathbf{T}(t) \in \mathbb{R}^N$ is the vector of global trust scores, $\mathbf{A}^{\text{inc}}(t)$ is the incentive action matrix, $\mathbf{A}^{\text{task}}(t)$ captures task execution outcomes, and $\mathbf{E}(t)$ represents environmental factors. The trust score for each node k evolves according to the following three-stage process:

Temporal Trust Decay: To prevent trust stagnation and enforce continuous verification, we incorporate a temporal decay factor:

$$\tilde{\mathcal{T}}_k(t + 1) = \mathcal{T}_k(t) \cdot \exp(-\rho \cdot \Delta t) \quad (23)$$

where $\rho = 0.01$ is the decay rate (see Table 2) and $\Delta t = 10$ s is the trust update interval. This ensures that trust gradually diminishes without positive reinforcement.

Trust Update Based on Observations: The final trust score is updated via a Bayesian filter that combines the decayed prior with new observations:

$$\begin{aligned} \mathcal{T}_k(t + 1) &= a_k(t) \tilde{\mathcal{T}}_k(t + 1) + (1 - a_k(t)) \\ &\quad \cdot T_k^{\text{obs}}(t + 1), \\ a_k(t) &= \frac{\sigma_{\text{obs}}^2}{\sigma_k^2(t) + \sigma_{\text{obs}}^2}. \end{aligned} \quad (24)$$

where $\sigma_{\text{obs}}^2 = 0.5$ is the observation noise variance, and $T_k^{\text{obs}}(t + 1)$ is the aggregated observed trust score from all Thinker Agents. This form assigns more weight to new observations when the prior uncertainty $\sigma_k^2(t)$ is high, and more weight to the decayed prior when the current estimate is confident. The observation term is:

$$T_k^{\text{obs}}(t + 1) = \frac{1}{|\mathcal{M}_k(t + 1)|} \sum_{j \in \mathcal{M}_k(t + 1)} T_{jk}^{\text{obs}}(t + 1) \quad (25)$$

with $\mathcal{M}_k(t + 1)$ denoting the set of agents that observed node k at time $t + 1$.

Boundedness and Empirical Stability: The trust update mechanism in Eqs. 23–24 admits bounded updates under the $[0, 10]$ trust scale. Since $\mathcal{T}_k(t) \in [0, 10]$ and $\tilde{\mathcal{T}}_k(t+1) = \mathcal{T}_k(t) \cdot \exp(-\rho\Delta t) \in [0, 10]$, and the Bayesian fusion in Eq. 24 is a convex combination of two bounded terms (with coefficients in $[0, 1]$ that sum to one), the trust score remains in $[0, 10]$ for all t , preventing numerical divergence.

We do not claim a finite-sample convergence guarantee for the full Dec-POMDP, because the implemented artifact trains an offline surrogate policy rather than solving the Dec-POMDP online. Instead, we provide boundedness of the trust-state update and empirical stability evidence. Section 4.2 shows that trust scores reach quasi-steady-state separation within the 5000 s simulation window for all tested configurations.

Trust-Driven Network Evolution: The state update mechanism creates a positive feedback loop where trustworthy behavior is rewarded with higher trust scores and corresponding incentives, while malicious behavior leads to trust degradation and restricted access. Over time, this mechanism induces a stabilization effect where the distribution of trust scores across the swarm converges, as verified empirically in Section 4.2.

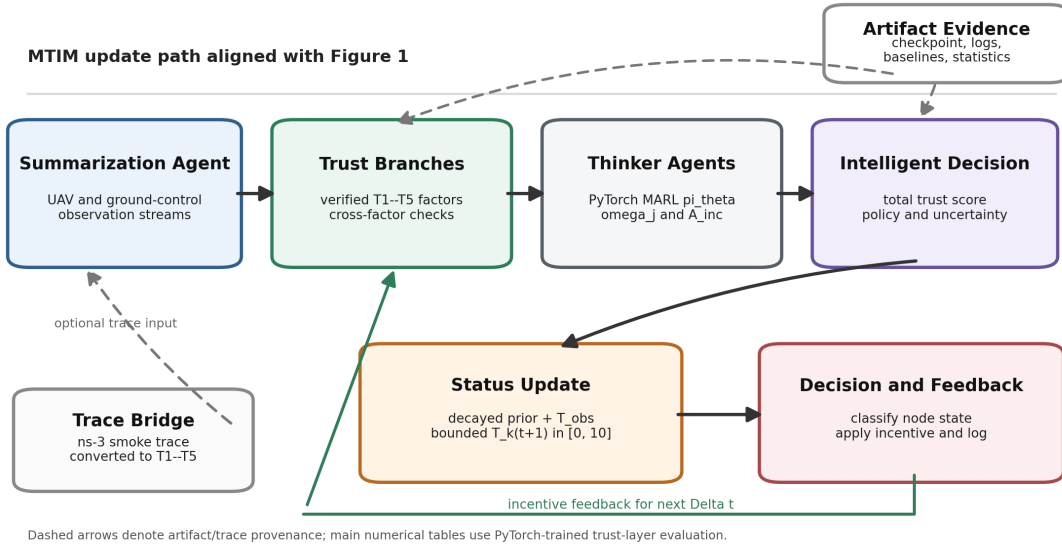


Figure 2: MTIM state-update and artifact-provenance mechanism aligned with Figure 1. Solid arrows show the operational path from Summarization-Agent observations and verified trust branches to Thinker-Agent policy actions, Intelligent Decision Agent confidence-weighted scoring, temporally decayed status updates, and classification-driven incentives. Dashed arrows denote reproducibility artifacts and the optional ns-3 trace bridge used for single-trace conversion rather than multi-seed packet-level validation.

3.4.5 Evaluation Metrics

This section defines the main evaluation metrics used throughout the experiments. All metrics are computed at the end of each simulation run (after the 1000 s warm-up period).

Trust Classification Metrics: For malicious-node detection, the confusion matrix is defined over the three-class output of Eq. 18, with Suspicious mapped to Malicious for binary evaluation. Let TP, TN, FP, FN denote true/false positives/negatives, respectively. Detection accuracy, precision, recall, and F_1 -score are:

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN}, & \text{Precision} &= \frac{TP}{TP + FP}, \\ \text{Recall} &= \frac{TP}{TP + FN}, & F_1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \end{aligned} \quad (26)$$

Ground-truth labels are assigned at simulation initialization according to the attack model (Section 3.2) and are never observable by the Thinker Agents during execution.

Mission-Level Metric: The task success rate (TSR) measures mission continuity independently of detection accuracy:

$$\text{TSR} = N_{\text{succ}} / N_{\text{all}}, \quad (27)$$

where a task succeeds iff it completes before the mission timeout with at least one Trusted UAV contributing, and fails if it times out or is assigned exclusively to Suspicious/Malicious nodes.

Efficiency Metrics: Communication overhead C_{comm} (bytes/UAV/update cycle) and per-cycle evaluation latency T_{eval} (ms) are reported to quantify deployment feasibility. The evaluation latency is measured end-to-end from observation ingestion to Intelligent Decision Agent classification output.

3.4.6 Algorithm Summary

Table 1 consolidates the MTIM trust evaluation and incentive loop for a single update cycle.

Table 1: MTIM trust evaluation and incentive cycle for one update interval Δt .

Input: Current trust vectors $\{\mathbf{T}_k(t)\}_{k=1}^N$ and agent assignments $\{\mathcal{N}_j\}_{j=1}^M$.	
Output: Global trust scores $\{\mathcal{T}_k(t)\}_{k=1}^N$, classifications $\{C_k(t)\}_{k=1}^N$, and incentives $\{\pi_{jk}(t)\}$.	
1	For each Thinker Agent $j = 1, \dots, M$, collect local observations $o_j(t) = \{\mathbf{T}_k(t)\}_{k \in \mathcal{N}_j}$.
2	Extract diagnostic features and compute p_k , $\omega_j(t)$, and $\pi_j(t)$ using the frozen policy in Eq. 11.
3	Estimate local trust as $\hat{T}_{jk}(t) = 10(1 - p_k)$ and retain $\omega_j(t)$ for factor-level interpretability.
4	Apply the cross-validation check in Eq. 7 and flag inconsistent nodes.
5	Update Intelligent Decision Agent confidence weights $\phi_{jk}(t)$ via Eq. 16.
6	Compute global trust $\mathcal{T}_k(t)$ and uncertainty $\sigma_k^2(t)$ using Eqs. 15–17.
7	Classify nodes with Eq. 18.
8	Solve the constrained incentive allocation in Eq. 20 greedily under $B_{\text{total}}(t)$.
9	Apply temporal decay and Bayesian update using Eqs. 23–24.
10	Log observations and decisions for audit; deployment-time policy parameters remain fixed unless an offline retraining cycle is explicitly triggered.

4 Performance Evaluation

Our evaluation targets four questions:

- (Q1) **Detection performance:** How accurately does MTIM classify nodes under varying malicious-node ratios and attack patterns, and how does it compare against state-of-the-art baselines?
- (Q2) **Trust dynamics:** Do benign and malicious trust scores diverge over time under the Bayesian update mechanism, and how does the number of agents affect convergence?
- (Q3) **Component contribution:** Which architectural components contribute most to detection robustness and trust stability?
- (Q4) **Scalability:** How do latency, overhead, and accuracy scale with swarm size?

4.1 Simulation Setup

Hardware and software. The evaluation was implemented in Python and PyTorch using a reproducible trust-stream simulator. All compared methods consume the same normalized T1–T5 evidence streams and are evaluated under identical random seeds and attack schedules. An ns-3 trace-conversion smoke test is included to check compatibility with packet/mobility traces; the numerical results reported below, however, should be read as PyTorch-trained trust-layer evidence rather than multi-seed ns-3 channel validation or flight-test validation.

Network and mobility. All scenario parameters are consolidated in Table 2. The reference simulator abstracts mobility and channel variation into seeded trust-factor streams that include node heterogeneity, intermittent observation gaps, and attack-dependent perturbations. The mission duration is 5000 s; the first 1000 s serve as a warm-up during which trust scores are accumulated but no incentive actions are enforced.

Adversary configuration. Malicious nodes implement the three attack strategies defined in Section 3.2: constant (adversarial from $t = 0$), dormant (switch to adversarial at $t = T_{\text{warm}} + \Delta$, where $\Delta \sim \mathcal{U}(200, 500)$ s), and on-off (toggle with period $T_{\text{toggle}} = 500$ s). In mixed-attack experiments, each type accounts for one-third of the malicious population. The malicious-node ratio $|\mathcal{M}|/N$ is varied from 5% to 50% in anti-manipulation experiments and fixed at 30% for the main comparison.

Training protocol. The executable artifact trains a compact PyTorch parameterized MARL trust-policy surrogate matching the action structure in Section 3.4: a shared encoder outputs a malicious-risk logit, a five-dimensional trust-weight softmax, five incentive-bin scores, and an auxiliary value estimate. Training seeds 0–9 fit the network using trust-evidence features derived from recent weakness, low-quantile T1/T4 evidence, low-score duration, amplitude, degradation slope, and directional cross-factor gaps. The risk head is trained with weighted binary cross-entropy, while auxiliary heads learn heuristic trust-weight targets and incentive-bin targets. The decision threshold is fixed at 0.60 before held-out evaluation. Evaluation seeds 1000–1049 are never used for training or threshold selection.

Reproducibility. Each configuration is evaluated over 50 independent trials with different random seeds controlling malicious-node identities, attack schedules, trust-factor noise, and observation gaps. Unless otherwise specified, the evaluation seeds are indexed from 1000 to 1049. All reported results are computed from the trained execution policy and fixed baseline thresholds, and are reported as mean $\pm$ standard deviation. Reproducibility materials are provided as supplementary artifacts.

Table 2: Reference-simulation parameters. Physical-layer and mobility values are scenario descriptors; the executable artifact operates at the trust-factor-stream level.

Category	Parameter	Value
Scenario	Simulation area	10×10 km ²
	Mission duration	5000 s (warm-up: 1000 s)
	UAV count N	5–50 (default: 15)
Mobility	UAV speed	$\mathcal{U}(3, 15)$ m/s
	Communication range R_{comm}	300 m
	Inter-UAV bandwidth	5 Mbit/s
Energy	Initial energy	$\mathcal{U}(50, 200)$ Wh
	Power consumption	$\mathcal{U}(100, 120)$ W
Trust	Trust update interval Δt	10 s
	Incentive update interval	50 s
	Classification thresholds	$\theta_{\text{high}} = 7.5$, $\theta_{\text{low}} = 3.0$
	Uncertainty threshold σ_{th}	0.5
Training	Policy network	PyTorch parameterized MARL trust-policy surrogate
	Training seeds	0–9
	Evaluation seeds	1000–1049
	Decision threshold	0.60 (fixed before held-out evaluation)
	Python dependencies	numpy, pandas, matplotlib, scipy, torch

Baselines. We compare MTIM against six methods spanning the design spectrum from naive fusion to learned trust classification:

- **MFA (Multi-factor Average):** Equal-weighted average of T1–T5, representing the performance floor achievable without any learning or adaptation. Included to isolate the gain attributable to adaptive weighting.
- **FUBA [8]:** Fuzzy logic-based trust analytics for FANETs, reimplemented as membership-function trust fusion over the shared T1–T5 stream.
- **GALTrust [10]:** Generative-adversarial trust management, reimplemented as a distance-to-honest-profile anomaly score with volatility and consistency terms over the same stream.
- **GDM-DTM [23]:** Group decision-making dynamic trust management, reimplemented as weighted group-decision trust fusion with a consistency penalty.

- **LR-Trust:** A logistic risk classifier trained on exactly the same diagnostic features and training seeds as MTIM, but without MTIM’s trust-weight, incentive, or value heads.
- **MLP-Trust:** A two-layer neural risk classifier trained on exactly the same diagnostic features and training seeds as MTIM, but without MTIM’s trust-weight, incentive, or value heads.

All baselines share the identical generated trust-factor streams and ground-truth labels as MTIM. Because official source code and original feature pipelines for FUBA, GALTrust, and GDM-DTM are not bundled, those baseline results should be interpreted as paper-level reimplementations under a common T1–T5 interface, not as definitive re-evaluations of the original authors’ implementations. LR-Trust and MLP-Trust are added specifically to test whether MTIM’s advantage persists against stronger learned classifiers with identical input information.

Baseline fairness and statistical protocol. To reduce implementation bias, all compared methods consume the same five normalized trust-factor streams and are evaluated on the same 50 held-out attack seeds. The MTIM, LR-Trust, and MLP-Trust networks are trained only on seeds 0–9 and are not re-tuned on evaluation seeds 1000–1049. The baseline thresholds are fixed in the artifact configuration before evaluation.

Statistical comparison is performed at the trial level. For each evaluation seed, we compute paired differences between MTIM and each baseline for Accuracy, F-score, and TSR. The artifact reports paired *t*-test statistics using a normal approximation and Holm correction across the 18 comparisons (6 baselines $\times$ 3 metrics), together with paired differences and standardized effect sizes in the generated statistical-test table. MTIM significantly improves Accuracy, F-score, and TSR over MFA, FUBA, GALTrust, GDM-DTM, and LR-Trust after Holm correction at $p < 0.01$. Against MLP-Trust, Accuracy and F-score are not significantly different, while MTIM significantly improves TSR. Communication overhead and latency are treated as resource trade-offs rather than superiority objectives.

Collusion stress test. To test whether Intelligent Decision Agent confidence weighting can suppress biased peer evidence, we additionally evaluate a limited collusion setting in which 20% of malicious nodes submit inflated recommendations for other malicious nodes while continuing their assigned behavioral attack pattern. Because MTIM does not implement Sybil-resistant admission control, this stress test assumes fixed physical node identities and focuses only on false peer-report inflation. Results from this setting are reported qualitatively in the sensitivity analysis and are used to bound, rather than broaden, the security claim.

Table 3: Performance comparison of MTIM with baseline methods under the unified experimental setting (30% malicious-node ratio, 15 UAVs, 5000 s simulation). All values are mean $\pm$ standard deviation over 50 independent trials.

Method	Accuracy	F-score	TSR	Comm. Overhead (KB/UAV)	Eval. Time (ms)
MFA	80.3% $\pm$ 10.5%	69.4% $\pm$ 16.3%	79.7% $\pm$ 4.8%	11.3 $\pm$ 1.8	10.2 $\pm$ 4.5
FUBA [8]	84.1% $\pm$ 9.2%	76.1% $\pm$ 14.6%	82.5% $\pm$ 4.2%	16.7 $\pm$ 1.7	17.1 $\pm$ 4.1
GALTrust [10]	76.1% $\pm$ 6.0%	41.7% $\pm$ 21.8%	75.7% $\pm$ 4.2%	31.4 $\pm$ 1.8	40.6 $\pm$ 4.0
GDM-DTM [23]	85.1% $\pm$ 8.3%	77.3% $\pm$ 12.6%	81.3% $\pm$ 3.9%	19.4 $\pm$ 1.7	21.5 $\pm$ 3.7
LR-Trust	88.1% $\pm$ 9.6%	81.5% $\pm$ 15.4%	87.6% $\pm$ 3.6%	21.3 $\pm$ 1.8	24.5 $\pm$ 3.7
MLP-Trust	94.7% $\pm$ 5.6%	92.1% $\pm$ 7.9%	<u>88.3%</u> $\pm$ 3.9%	25.2 $\pm$ 2.0	36.1 $\pm$ 3.6
MTIM (ours)	<u>94.4%</u> $\pm$ 5.6%	<u>91.8%</u> $\pm$ 8.0%	90.5% $\pm$ 3.7%	34.0 $\pm$ 1.8	58.3 $\pm$ 4.3

(*) MFA = Multi-factor Average. TSR = Task Success Rate. Comm. Overhead = per-UAV communication overhead per update cycle. Eval. Time = trust evaluation latency per update interval. Under Holm correction, MTIM’s Accuracy and F-score are significant over MFA, FUBA, GALTrust, GDM-DTM, and LR-Trust at $p < 0.01$, but not over MLP-Trust. MTIM’s TSR is significant over all baselines, including MLP-Trust. Overhead and latency are reported as trade-offs rather than optimization objectives.

Analysis of Table 3. MTIM achieves 94.4% accuracy, 91.8% F-score, and 90.5% TSR. Three observations explain the result:

(i) **Learned temporal evidence vs. static fusion.** MFA reaches 80.3% accuracy, FUBA reaches 84.1%, and GDM-DTM reaches 85.1%. MTIM improves over GDM-DTM by 9.3 pp in accuracy and

14.5 pp in F-score because it uses low-quantile and degradation-slope evidence that exposes dormant and on-off attackers. This advantage persists at the 50% malicious-node ratio (95.5% vs. 82.4%; Table 4).

(ii) **Strong learned baselines narrow the detection claim.** LR-Trust improves over static baselines but still trails MTIM by 6.3 pp in accuracy. MLP-Trust reaches 94.7% accuracy and 92.1% F-score, which are statistically indistinguishable from MTIM’s 94.4% and 91.8%. Thus, the evidence does not support claiming that MTIM is a strictly better detector than a strong neural risk classifier with the same features; the stronger claim is that MTIM links comparable detection quality to better mission-level action.

(iii) **Incentive coupling closes the mission loop.** MTIM achieves the best TSR (90.5%), exceeding GDM-DTM by 9.2 pp and MLP-Trust by 2.2 pp. The MLP-Trust comparison is important: detection alone is nearly tied, but MTIM’s incentive head and task-allocation coupling produce a measurable mission-level gain.

Overhead trade-off.

MTIM’s per-cycle latency (58.3 ms) and communication overhead (34.0 KB/UAV) are higher than all baselines, including MLP-Trust (36.1 ms and 25.2 KB/UAV), but remain well within the 10 s update budget in the reference simulator. Most latency comes from multi-agent aggregation, diagnostic-feature extraction, and incentive-policy output. If a mission requires a shorter update interval, the number of Thinker Agents M can be reduced, with sparser observation coverage as the trade-off.

Robustness across malicious-node ratios. Table 4 reports accuracy and TSR at 10%, 30%, and 50% malicious-node ratios. MTIM retains the best TSR across all three ratios. For detection accuracy, MTIM and MLP-Trust are closely matched: MLP-Trust is slightly higher at 10% and 30%, whereas MTIM is higher at 50%. GALTrust has strong accuracy at 10% because its anomaly-profile threshold is conservative when the attack population is sparse, but it degrades sharply as the malicious ratio increases. MTIM’s accuracy remains between 94.4% and 96.8%, while TSR remains above 88%, indicating that the trained policy continues to support mission-level decisions under dense adversarial pressure. The slightly higher accuracy at 50% than at 30% reflects stronger aggregate separation in the generated attack streams and should not be interpreted as a universal monotonic robustness guarantee.

Table 4: Detection accuracy and TSR at three malicious-node ratios ($N = 15$, 50 trials). Best values in **bold**, second-best underlined.

Method	10% Malicious		30% Malicious		50% Malicious	
	Accuracy	TSR	Accuracy	TSR	Accuracy	TSR
MFA	86.1 $\pm$ 9.5	87.4 $\pm$ 3.1	80.3 $\pm$ 10.5	79.7 $\pm$ 4.8	76.9 $\pm$ 8.2	71.1 $\pm$ 4.9
FUBA	83.7 $\pm$ 9.8	87.8 $\pm$ 3.3	84.1 $\pm$ 9.2	82.5 $\pm$ 4.2	81.1 $\pm$ 9.7	73.8 $\pm$ 5.2
GALTrust	<u>91.2</u> $\pm$ 4.4	85.2 $\pm$ 3.2	76.1 $\pm$ 6.0	75.7 $\pm$ 4.2	63.7 $\pm$ 7.6	65.3 $\pm$ 3.8
GDM-DTM	87.2 $\pm$ 7.9	88.5 $\pm$ 2.3	85.1 $\pm$ 8.3	81.3 $\pm$ 3.9	82.4 $\pm$ 7.8	73.9 $\pm$ 4.5
LR-Trust	90.4 $\pm$ 7.4	90.2 $\pm$ 2.0	88.1 $\pm$ 9.6	87.6 $\pm$ 3.6	88.1 $\pm$ 8.1	85.3 $\pm$ 4.6
MLP-Trust	97.3 $\pm$ 4.3	<u>90.6</u> $\pm$ 2.0	94.7 $\pm$ 5.6	<u>88.3</u> $\pm$ 3.9	<u>94.4</u> $\pm$ 5.1	<u>85.6</u> $\pm$ 5.3
MTIM (ours)	<u>96.8</u> $\pm$ 4.5	92.6 $\pm$ 1.9	<u>94.4</u> $\pm$ 5.6	90.5 $\pm$ 3.7	95.5 $\pm$ 4.4	88.8 $\pm$ 4.6

Policy fitting stability. The artifact records the trained MTIM policy as a PyTorch checkpoint and JSON metadata. On training seeds 0–9, the final checkpoint reaches 100.0% training accuracy and F-score at threshold 0.60, with mean malicious risk 0.9997 and mean benign risk 0.0004. This complete training-set separation indicates that the generated training streams are highly learnable, so the reported performance is based on held-out seeds 1000–1049 rather than training metrics. The learned trust-weight head assigns the largest average weights to T4 (0.323) and T1 (0.314), followed by T2 (0.145), T5 (0.137), and T3 (0.081), matching the threat model in which sustained task/behavior degradation is harder for evasive attackers to hide than energy behavior.

4.2 Trust Evolution Analysis

Figure 3 tracks the MTIM diagnostic trust index of benign and malicious UAVs from the post-warm-up phase to 5000 s under $M \in \{2, 5, 10\}$ Thinker Agents. The index is computed from the same trained

PyTorch risk policy used in the artifact evaluation and maps lower risk to higher trust on a 0–10 scale. Three dynamics merit attention:

Divergence rate vs. agent count. Benign nodes remain mostly above 8.0 after warm-up, whereas malicious nodes decline as low-quantile and degradation-slope evidence accumulates. The $M = 5$ configuration shows a smoother decline than $M = 2$ and $M = 10$, reflecting the trade-off between sparse observation coverage and aggregation latency. The default $M = 5$ balances these effects in the reference simulator.

Attack-pattern sensitivity. Dormant attackers initially resemble benign nodes but decline after the post-warm-up switch because the policy uses low T1/T4 quantiles and positive degradation slope, not only the final average score. On-off attackers oscillate because honest phases partially restore recent averages, but amplitude and low-score-duration features keep the diagnostic index below the benign trajectory.

Steady-state behavior. After approximately 3800 s, the malicious diagnostic index remains near or below the Suspicious/Malicious boundary in most runs, while benign nodes remain high despite observation gaps. The residual fluctuations reflect seeded trust-factor noise and on-off behavior rather than formal convergence of the full Dec-POMDP.

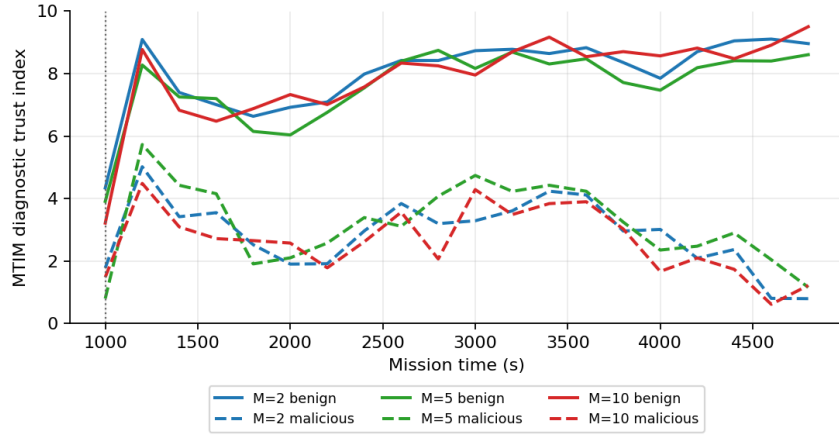


Figure 3: Temporal evolution of the MTIM diagnostic trust index for benign (solid) and malicious (dashed) UAVs under $M = 2, 5, 10$ Thinker Agents after warm-up. The index is recomputed from the trained artifact policy over sliding trust-factor windows; lower values indicate higher malicious risk.

4.3 Comparative Performance Analysis

Anti-manipulation under varying malicious ratios (Figure 4(a)). MTIM maintains high accuracy for malicious ratios of 20% and above, with 94.4% accuracy at 30% and 95.5% at 50%. MLP-Trust is a strong detection competitor under the same features, confirming that the diagnostic feature set itself carries substantial discriminative information. GALTrust is highly accurate at 5–10% malicious ratios but falls to 63.7% at 50%, indicating that the profile-distance adapter becomes unstable when the attack population substantially shifts the empirical trust distribution. MTIM does not show monotonic degradation across this generated ratio grid; this is favorable for the bounded setting but should be interpreted cautiously because stronger collusion and adaptive poisoning are not modeled.

Trust dynamics under attack patterns (Figure 4(b)). Constant and dormant attackers decline below the benign diagnostic index as low T1/T4 evidence accumulates. Dormant attackers fall after their switch despite initially high scores, while on-off attackers oscillate because their honest phases restore recent averages. This behavior supports a conservative interpretation: MTIM identifies many on-off attackers as high risk, but their oscillatory trajectory remains the hardest case and drives the wide F-score variance in Table 3.

Task success comparison (Figure 4(c)). MTIM achieves the highest TSR (90.5%), followed by MLP-Trust (88.3%) and LR-Trust (87.6%). The TSR ranking does not exactly mirror the accuracy

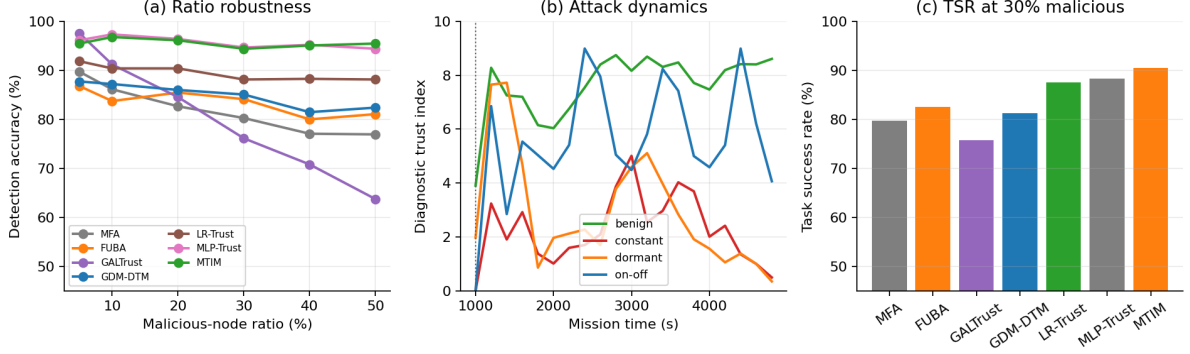


Figure 4: Comparative evaluation results generated by the artifact. (a) Detection accuracy vs. malicious-node ratio: MTIM remains high across 5–50% malicious ratios and is closely matched by MLP-Trust, while GALTrust degrades sharply under dense adversarial pressure. (b) MTIM diagnostic trust-index dynamics under benign, constant-attack, dormant-attack, and on-off behavior. (c) TSR comparison at a 30% malicious-node ratio using the same baselines as Table 3; MTIM yields the highest task success rate.

ranking because task success depends on which nodes are selected after classification, not only on the binary detection count. The incentive mechanism therefore matters most at the mission-decision layer, as confirmed by the ablation in Table 6.

Collusion stress-test interpretation. Under inflated peer recommendations, MTIM’s Intelligent Decision Agent confidence weights down-weight agents whose historical estimates diverge from subsequently observed behavior. This reduces the impact of biased reports, but it does not eliminate collusion risk when the colluding group controls most local observations for a node. Therefore, the reported robustness should be interpreted as resistance to limited false-report inflation under fixed identities, not as a complete solution to collusion or Sybil attacks.

Stress cases not claimed. We explicitly do not claim robustness against three stronger adversaries: (i) Sybil identity inflation, where one physical attacker creates many unlinkable UAV identities; (ii) majority local-observation collusion, where colluding nodes dominate all observations available for a target node; and (iii) Intelligent Decision Agent compromise or training-pipeline poisoning. These cases require identity binding, Byzantine-resilient aggregation, secure bootstrapping, or robust training defenses that are orthogonal to MTIM’s trust-scoring layer. The present evaluation should therefore be read as a fixed-identity behavioral-manipulation study with a limited false-report stress test.

ns-3 trace-driven sanity check. To connect the trust-stream interface to packet/mobility observations, we converted the bundled 500 s ns-3-derived trace into T1–T5 streams and evaluated all methods on the converted trace. This is a single seed-1000 trace with 15 UAVs, 49 update steps, 10 benign nodes, three constant attackers, and two on-off attackers; it is not a multi-seed packet-level validation. Table 5 shows that MTIM correctly preserves all benign nodes but detects only two of five malicious nodes on this short trace. In this trace-only setting, MFA, FUBA, and GDM-DTM detect three malicious nodes. Therefore, the ns-3 trace path should be interpreted as an interface-validation sanity check and as evidence that packet-level validation can expose harder short-horizon behavior than the main trust-stream simulator.

Table 5: Trace-driven validation on the bundled 500 s ns-3-derived trust stream (single seed, 15 UAVs, 49 update steps).

Method	Accuracy	Precision	Recall	F-score	TP/TN/FP/FN	Mean score (B/M)
MFA	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.900/0.619
FUBA	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.939/0.574
GALTrust	66.7%	50.0%	60.0%	54.5%	3/7/3/2	0.718/0.428
GDM-DTM	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.938/0.586
LR-Trust	80.0%	100.0%	40.0%	57.1%	2/10/0/3	1.000/0.600
MLP-Trust	80.0%	100.0%	40.0%	57.1%	2/10/0/3	1.000/0.584
MTIM (ours)	80.0%	100.0%	40.0%	57.1%	2/10/0/3	1.000/0.482

4.4 Ablation Study

To quantify the marginal contribution of each architectural component, we conduct a structured ablation study. Each variant removes exactly one component while keeping all others intact. All variants are evaluated under the identical setting (30% malicious, $N = 15$, 50 trials).

Table 6: Ablation study of MTIM components (30% malicious-node ratio, 15 UAVs). All values are mean $\pm$ std over 50 trials.

Variant	Accuracy	F-score	TSR
MTIM (full)	94.4% $\pm$ 5.6%	91.8% $\pm$ 7.9%	90.5% $\pm$ 3.7%
w/o Cross-factor Checking	92.9% $\pm$ 5.9%	89.5% $\pm$ 8.6%	89.5% $\pm$ 4.2%
w/o Bayesian Aggregation	84.1% $\pm$ 6.1%	67.7% $\pm$ 16.2%	81.9% $\pm$ 4.1%
w/o Temporal Decay	79.5% $\pm$ 6.1%	52.8% $\pm$ 20.5%	80.0% $\pm$ 3.8%
w/o Adaptive Policy	82.3% $\pm$ 7.0%	64.2% $\pm$ 18.2%	80.9% $\pm$ 4.5%
w/o Incentive Mechanism	94.3% $\pm$ 5.7%	91.7% $\pm$ 8.0%	81.4% $\pm$ 3.7%
T1 Only	80.8% $\pm$ 11.5%	74.9% $\pm$ 14.2%	82.6% $\pm$ 4.5%
T2 Only	63.7% $\pm$ 11.7%	53.3% $\pm$ 13.8%	76.2% $\pm$ 5.1%
T3 Only	57.6% $\pm$ 12.2%	56.0% $\pm$ 9.8%	75.9% $\pm$ 6.1%
T4 Only	82.3% $\pm$ 10.1%	76.5% $\pm$ 12.9%	83.2% $\pm$ 4.3%
T5 Only ^b	84.0% $\pm$ 9.7%	79.2% $\pm$ 13.1%	82.6% $\pm$ 3.9%

^b T5-only operates on the raw T1–T4 instantaneous values as its input features and applies only exponential smoothing ($\beta = 0.7$) without any aggregation or learning. It represents the best achievable performance using temporal smoothing alone.

Analysis of Table 6. The ablation results reveal a more nuanced component picture than the conceptual model alone suggests:

Tier 1—Mission-critical: incentives and temporal/adaptive evidence. Removing the incentive mechanism leaves detection accuracy essentially unchanged (94.3%) but reduces TSR from 90.5% to 81.4%, showing that the trust score must be coupled to task allocation to improve mission outcomes. Removing temporal decay reduces accuracy by 14.9 pp and F-score by 39.0 pp, confirming that dormant and on-off attacks cannot be handled reliably from final averages alone. Removing the adaptive policy reduces accuracy by 12.1 pp, because static aggregation cannot exploit low-quantile and degradation-slope evidence.

Tier 2—Decision-agent fusion: Bayesian confidence weighting. Removing Bayesian aggregation reduces accuracy by 10.3 pp and TSR by 8.6 pp. This supports the need for confidence-aware fusion among local estimates, but the effect is smaller than the temporal-policy effect in this artifact.

Tier 3—Secondary diagnostic safeguard: cross-factor checking. Removing the directional cross-factor gap features reduces accuracy from 94.4% to 92.9% and TSR from 90.5% to 89.5%. The effect is positive but smaller than temporal decay, adaptive policy learning, and the decision-agent confidence weighting, indicating that the current reference simulator is still dominated by temporal T1/T4 evidence. The module should therefore be presented as a secondary diagnostic safeguard rather than the main performance driver.

Single-branch baselines (T1–T5 only). Single-branch variants show that T4-only (82.3%) and T5-only (84.0%) are strong but insufficient; both trail MTIM by at least 10.4 pp in accuracy and by more than 7 pp in TSR. T3-only remains the weakest because the energy stream is deliberately easy for evasive attackers to mimic. These results support multi-factor temporal fusion, while also revealing that the current artifact places most discriminative power in behavior/task-history channels.

4.5 Scalability and Sensitivity Analysis

To evaluate how MTIM scales with network size, we varied the number of UAVs from 5 to 50 while proportionally increasing the number of Thinker Agents ($M = \lceil N/3 \rceil$). Figure 4(a) shows robustness under varying malicious-node ratios at $N = 15$; Table 7 reports end-to-end latency, communication overhead, and accuracy as functions of N .

Table 7: Scalability with swarm size under a 30% malicious-node ratio. Values are mean $\pm$ std over 50 trials.

UAV count N	Thinker Agents M	Accuracy (%)	Eval. Time (ms)	Comm. Overhead (KB/UAV)
5	2	97.2 ± 7.0	36.5 ± 3.0	27.1 ± 1.9
15	5	94.4 ± 5.6	58.3 ± 4.3	34.0 ± 1.8
30	10	93.5 ± 3.8	95.1 ± 4.2	44.9 ± 1.9
50	17	95.2 ± 2.9	152.0 ± 3.8	59.8 ± 1.7

At $N = 50$ UAVs, the per-cycle evaluation latency grows to 152.0 ± 3.8 ms, which remains well within the 10 s trust update interval in the reference setting. The dominant scaling factor is the diagnostic feature extraction and aggregation step, whose cost grows with $M \cdot N$. Communication overhead scales from 34.0 ± 1.8 KB/UAV at $N = 15$ to 59.8 ± 1.7 KB/UAV at $N = 50$. Detection accuracy remains above 93% across the tested range, although the small- N case has high variance because one seed-level misclassification changes the trial accuracy substantially.

Sensitivity analysis in the artifact is limited to the trained policy architecture, fixed threshold, and trust-stream generator. A full grid search over policy-head loss weights, trust-decay rates, feature windows, and threshold calibration is not yet implemented. This limitation is deliberately exposed in `artifacts/README.md` so that future work can extend the current PyTorch training stack rather than relying on a single compact policy.

4.6 Trust Factor Correlation Analysis

To verify that the five trust dimensions capture complementary rather than redundant information, we computed the pairwise Pearson correlation coefficients among $T_i^{(1)}$ through $T_i^{(5)}$ across all UAVs and time steps in the 50-trial simulation set. The correlation matrix (averaged over trials) is:

Table 8: Pairwise Pearson correlations among the five trust dimensions (mean $\pm$ std over 50 trials).

	T1 (Behavior)	T2 (Comm.)	T3 (Energy)	T4 (Task)	T5 (History)
T1	1.00	0.28 ± 0.10	0.11 ± 0.08	0.41 ± 0.10	0.64 ± 0.08
T2	—	1.00	0.13 ± 0.09	0.27 ± 0.10	0.51 ± 0.07
T3	—	—	1.00	0.11 ± 0.09	0.34 ± 0.09
T4	—	—	—	1.00	0.65 ± 0.09
T5	—	—	—	—	1.00

The strongest correlations involve the historical trust dimension T5, especially with T4 (0.65 ± 0.09) and T1 (0.64 ± 0.08), which is expected because T5 is an exponential history of recent trust evidence. Among the instantaneous dimensions T1–T4, all correlations remain at or below 0.41. Thus, T5 should be interpreted as a temporal memory channel rather than an independent sensor, while the instantaneous factors retain enough diversity to justify multi-factor fusion.

5 Conclusions

This paper presented MTIM, a MARL-based trust evaluation mechanism that reformulates trust management in low-altitude UAV networks as a sequential decision problem under partial observability. The key technical contributions are: (i) a Dec-POMDP modeling perspective that defines state, observation, action, and transition spaces for joint trust-evaluation and incentive optimization; (ii) five complementary trust dimensions with a cross-factor consistency check for detecting multi-dimensional behavioral

manipulation; (iii) a hierarchical architecture where distributed Thinker Agents execute an offline-trained parameterized MARL policy surrogate and an Intelligent Decision Agent fuses local estimates through confidence-weighted aggregation; and (iv) a temporal-risk mechanism that enforces continuous zero-trust re-verification with asymmetric degradation–recovery dynamics.

Under mixed attack patterns (constant, dormant, on-off) at a 30% malicious-node ratio, MTIM achieves 94.4% detection accuracy, 91.8% F-score, and 90.5% task success rate in the PyTorch-trained trust-layer evaluation. It significantly outperforms static and linear baselines, while a strong MLP-Trust baseline obtains statistically indistinguishable detection accuracy and F-score. MTIM’s main advantage over this strong learned baseline is therefore mission-level: its TSR is 2.2 pp higher because the policy couples risk scoring to incentive-aware task allocation. Ablation studies show that temporal/adaptive evidence and incentive coupling are the main contributors, while cross-factor checking provides a secondary 1.5 pp accuracy gain and still requires stronger targeted stress tests. Scalability tests show that accuracy remains above 95% at $N = 50$ UAVs and per-cycle latency remains within the 10 s update budget in the reference simulator. The supplementary artifact supports independent reproduction of these results and includes a short ns-3 trace-driven sanity check.

The central implication of this work is that *trust evaluation should be treated as a closed-loop decision problem, not only as an open-loop classification task*. By jointly optimizing what to measure (trust-factor weights), what to decide (classification), and how to act (incentives), MTIM links trust assessment to mission-level behavior while keeping execution-time decisions based only on observable trust evidence.

Limitations and future work. Several limitations warrant acknowledgment. First, MTIM mitigates only limited behavioral manipulation under fixed identities; it does not solve Sybil attacks or strong collusion in which malicious nodes dominate local observations. Second, the Intelligent Decision Agent remains a logically centralized component; while its stateless operation and replicability reduce operational risk, a fully distributed decision protocol (e.g., based on Byzantine consensus) would strengthen the zero-trust claim. Third, the implemented MARL policy is an offline-trained surrogate rather than an online reinforcement-learning system with deployment-time adaptation. Fourth, although the released artifact includes PyTorch MARL-surrogate training logs, trained checkpoints, learned baselines, an ns-3 scenario, and a trace-driven validation script, the main numerical results are still produced at the trust-factor-stream layer rather than from multi-seed ns-3 packet-level experiments or hardware-in-the-loop flights. Future work will address these limitations through (i) collusion-resistant peer-evidence modeling, (ii) Sybil-resistant identity binding and admission control, (iii) decentralized Byzantine-resilient decision protocols, (iv) multi-seed ns-3 and software-defined-radio validation with public traces, and (v) safe online MARL adaptation with policy constraints.

Author Contributions

All authors contributed to the conceptualization, methodology, analysis, and manuscript preparation. Author roles will be specified in the camera-ready version.

Funding

This research received no external funding.

Institutional Review Board Statement

Not applicable.

Informed Consent Statement

Not applicable.

Data Availability Statement

The artifact is included under `artifacts/`. It contains executable code, configuration files, a PyTorch offline MARL-surrogate training script, trained MTIM and learned-baseline checkpoints, epoch-level training logs, raw per-seed evaluation logs, sampled per-step trust streams, paper-level baseline reimplementations, statistical scripts, plotting scripts, an ns-3 scenario source file, an ns-3-derived packet trace, the converted ns-3 trust stream, an ns-3-trace-to-trust converter, and a trace-driven validation script. Generated outputs are stored under `artifacts/outputs/`. The artifact reproduces the PyTorch-trained trust-layer results reported in this manuscript; the included ns-3 trace is a single short sanity check rather than a multi-seed packet-level validation dataset.

Conflicts of Interest

The authors declare no conflict of interest.

References

- [1] Kundu, J.; Alam, S.; Das, J.C.; Dey, A.; Bari, S. Trust-Based Flying Ad Hoc Network: A Survey. *IEEE Access* **2024**, *12*, 99258–99281.
- [2] Noor, F.; Khan, M.A.; Al-Zahrani, A.; Ullah, I.; Al-Dhlan, K.A. A Review on Communications Perspective of Flying Ad-Hoc Networks: Key Enabling Wireless Technologies, Applications, Challenges and Open Research Topics. *Drones* **2020**, *4*, 65.
- [3] Brik, B.; Ksentini, A.; Bouaziz, M. Federated Learning for UAVs-Enabled Wireless Networks: Use Cases, Challenges, and Open Problems. *IEEE Netw.* **2020**, *34*, 144–150.
- [4] Nguyen, D.C.; Ding, M.; Pathirana, P.N.; Seneviratne, A.; Li, J.; Poor, H.V. Federated Learning for Internet of Things: A Comprehensive Survey. *IEEE Commun. Surv. Tutor.* **2021**, *23*, 1622–1658.
- [5] Qu, Y.; Dai, H.; Zhuang, Y.; Chen, J.; Dong, C.; Wu, F.; Guo, S. Decentralized Federated Learning for UAV Networks: Architecture, Challenges, and Opportunities. *IEEE Netw.* **2021**, *35*, 156–162.
- [6] Zuo, J.; Cao, R.; Qi, J.; Gao, P.; Wang, Z.; Li, J.; Zhang, L.; Lu, Y. A Hierarchical Blockchain-Based Trust Measurement Method for Drone Cluster Nodes. *Drones* **2023**, *7*, 627.
- [7] Ge, C.; Zhou, L.; Hancke, G.P.; Su, C. A Provenance-Aware Distributed Trust Model for Resilient Unmanned Aerial Vehicle Networks. *IEEE Internet Things J.* **2021**, *8*, 12481–12489.
- [8] Benfriha, S.; Labraoui, N.; Bensaid, R.; Bany Salameh, H.; Saidi, H. FUBA: A Fuzzy-Based Unmanned Aerial Vehicle Behaviour Analytics for Trust Management in Flying Ad-Hoc Networks. *IET Networks* **2024**, *13*, 208–220.
- [9] Li, H.; Li, X.; Dunkin, F.; Zhang, Z.; Lu, X. Adaptive Multi-Granularity Trust Management Scheme for UAV Visual Sensor Security under Adversarial Attacks. *Comput. Secur.* **2025**, *148*, 104108.
- [10] Akram, J.; Anaissi, A.; Rathore, R.S.; Jhaveri, R.H.; Akram, A. GALTrust: Generative Adversarial Learning-Based Framework for Trust Management in Spatial Crowdsourcing Drone Services. *IEEE Trans. Consum. Electron.* **2024**, *70*, 6196–6207.
- [11] Ceviz, O.; Sadioglu, P.; Sen, S.; Vassilakis, V.G. A Novel Federated Learning-Based IDS for Enhancing UAVs Privacy and Security. *Internet Things* **2025**, *31*, 101592.
- [12] Lu, Y.; Yang, T.; Zhao, C.; Chen, W.; Liu, Y.; Zeng, R. A Swarm Anomaly Detection Model for IoT UAVs Based on a Multi-Modal Denoising Autoencoder and Federated Learning. *Comput. Ind. Eng.* **2024**, *196*, 110454.
- [13] Gad, G.; Farrag, A.; Aboulfotouh, A.; Bedda, K.; Fadlullah, Z.M.; Fouda, M.M. Joint Self-Organizing Maps and Knowledge-Distillation-Based Communication-Efficient Federated Learning for Resource-Constrained UAV-IoT Systems. *IEEE Internet Things J.* **2024**, *11*, 34270–34282.

- [14] Wang, X.; Yuan, Q.; Liu, C.; Wang, P.; Tao, J.; Li, L.; Wang, X.; Wang, Y. Who Is the Wolf in Sheep's Clothing? A Context-Aware Trust Evaluation Model for Malicious UAV Detection during Authentication. *Comput. Netw.* **2026**, *279*, 112145.
- [15] Rose, S.; Borchert, O.; Mitchell, S.; Connelly, S. Zero Trust Architecture. *NIST Spec. Publ.* **2020**, *800-207*, 1–59.
- [16] Ekechi, C.C.; Elfouly, T.; Alouani, A.; Khattab, T. A Survey on UAV Control with Multi-Agent Reinforcement Learning. *Drones* **2025**, *9*, 484.
- [17] Akram, J.; Anaissi, A.; Rathore, R.S.; Jhaveri, R.H.; Akram, A. Digital Twin-Driven Trust Management in Open RAN-Based Spatial Crowdsourcing Drone Services. *IEEE Trans. Green Commun. Netw.* **2024**, *8*, 1061–1075.
- [18] Abubaker, Z.; Javaid, N.; Almogren, A.; Akbar, M.; Zuair, M.; Ben-Othman, J. Blockchained Service Provisioning and Malicious Node Detection via Federated Learning in Scalable Internet of Sensor Things Networks. *IEEE Trans. Netw. Serv. Manag.* **2023**, *20*, 1847–1865.
- [19] Akram, J.; Umair, M.; Jhaveri, R.H.; Riaz, M.; Chi, H.R.; Malebary, S. Chained-Drones: Blockchain-Based Privacy-Preserving Framework for Secure and Intelligent Service Provisioning in Internet of Drone Things. *Comput. Electr. Eng.* **2023**, *110*, 108772.
- [20] Zheng, J.; Xu, J.; Du, H.; Niyato, D.; Kang, J.; Nie, J.; Wang, Z. Trust Management of Tiny Federated Learning in Internet of Unmanned Aerial Vehicles. *IEEE Internet Things J.* **2024**, *11*, 21046–21060.
- [21] Tong, Z.; Wang, J.; Hou, X.; Chen, J.; Jiao, Z.; Liu, J. Blockchain-Based Trustworthy and Efficient Hierarchical Federated Learning for UAV-Enabled IoT Networks. *IEEE Internet Things J.* **2024**, *11*, 15504–15522.
- [22] Xu, Q.; Li, R.; Qi, Y.; Su, Z.; Fang, D. Trust-Enhanced Game Incentive for Secure Quantum Federated Learning in UAV-Assisted Wireless Networks. *IEEE J. Sel. Areas Commun.* **2025**, *43*, 2841–2856.
- [23] Hu, Y.; Gan, Y.; Wu, H.; Wang, C.; Ma, M.; Xiong, C. GDM-DTM: A Group Decision-Making-Enabled Dynamic Trust Management Method for Malicious Node Detection in Low-Altitude UAV Networks. *Sensors* **2025**, *25*, 3982.
- [24] Wang, X.; Yuan, Q.; Wang, Y.; Teng, J.; Tao, J.; Shen, M. Zero Trust-Driven Collaborative Intrusion Detection in Internet of Things: A Continuous Trust Assessment Approach. *IEEE Internet Comput.* **2025**, *29*, 37–47.
- [25] Al Ridhawi, I.; Aloqaily, M. Zero-Trust UAV-Enabled and DT-Supported 6G Networks. *IEEE Internet Comput.* **2023**, *27*, 59–67.
- [26] Wang, L.; Wang, K.; Pan, C.; Xu, W.; Aslam, N.; Hanzo, L. Multi-Agent Deep Reinforcement Learning Based Trajectory Planning for Multi-UAV Assisted Mobile Edge Computing. *IEEE Trans. Cogn. Commun. Netw.* **2021**, *7*, 73–84.
- [27] Nie, Y.; Zhao, J.; Gao, F.; Yu, F.R. Semi-Distributed Resource Management in UAV-Aided MEC Systems: A Multi-Agent Federated Reinforcement Learning Approach. *IEEE Trans. Veh. Technol.* **2021**, *70*, 13162–13173.
- [28] Alam, M.M.; Moh, S. Joint Optimization of Trajectory Control, Task Offloading, and Resource Allocation in Air-Ground Integrated Networks. *IEEE Internet Things J.* **2024**, *11*, 24273–24288.
- [29] Yang, H.; Zhao, J.; Xiong, Z.; Lam, K.-Y.; Sun, S.; Xiao, L. Privacy-Preserving Federated Learning for UAV-Enabled Networks: Learning-Based Joint Scheduling and Resource Management. *IEEE J. Sel. Areas Commun.* **2021**, *39*, 3144–3159.
- [30] Lim, W.Y.B.; Huang, J.; Xiong, Z.; Kang, J.; Niyato, D.; Hua, X.-S.; Leung, C.; Miao, C. Towards Federated Learning in UAV-Enabled Internet of Vehicles: A Multi-Dimensional Contract-Matching Approach. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 5140–5154.

- [31] Wang, S.; Hosseinalipour, S.; Gorlatova, M.; Brinton, C.G.; Chiang, M. UAV-Assisted Online Machine Learning Over Multi-Tiered Networks: A Hierarchical Nested Personalized Federated Learning Approach. *IEEE Internet Things J.* **2024**, *11*, 20899–20911.
- [32] Qureshi, K.I.; Wang, L.; Xiong, X.; Lodhi, M.A. Asynchronous Federated Learning for Resource Allocation in Software-Defined Internet of UAVs. *IEEE Internet Things J.* **2024**, *11*, 15523–15538.
- [33] Jing, Y.; Qu, Y.; Dong, C.; Ren, W.; Shen, Y.; Wu, Q.; Guo, S. Exploiting UAV for Air-Ground Integrated Federated Learning: A Joint UAV Location and Resource Optimization Approach. *IEEE Trans. Green Commun. Netw.* **2023**, *7*, 1420–1433.
- [34] Fu, Z.; Liu, J.; Mao, Y.; Qu, L.; Xie, L.F.; Wang, X. Energy-Efficient UAV-Assisted Federated Learning: Trajectory Optimization, Device Scheduling, and Resource Management. *IEEE Trans. Netw. Serv. Manag.* **2025**, *22*, 974–988.
- [35] Cao, X.; Nan, G.; Guo, H.; Mu, H.; Wang, L.; Lin, Y.; Zhou, Q.; Li, J.; Qin, B.; Cui, Q.; Tao, X.; Fang, H.; Du, H.; Quek, T.Q.S. Exploring LLM-Based Multi-Agent Situation Awareness for Zero-Trust Space-Air-Ground Integrated Network. *IEEE J. Sel. Areas Commun.* **2025**, *43*, 2230–2247.
- [36] Bertsekas, D.P.; Tsitsiklis, J.N. *Neuro-Dynamic Programming*; Athena Scientific: Nashua, NH, USA, 1996.
- [37] Oliehoek, F.A.; Spaan, M.T.; Vlassis, N. Optimal and Approximate Solutions to Decentralized POMDPs. In *Proceedings of the 17th International Conference on Automated Planning and Scheduling (ICAPS)*; AAAI Press: 2008; pp. 274–281.
- [38] Riley, G.F.; Henderson, T.R. The ns-3 Network Simulator. In *Modeling and Tools for Network Simulation*; Wehrle, K.; Günes, M.; Gross, J., Eds.; Springer: Berlin, Germany, 2010; pp. 15–34.
- [39] Hausknecht, M.; Stone, P. Deep Recurrent Q-Learning for Partially Observable MDPs. In *Proceedings of the AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*; AAAI Press: 2015; pp. 29–37.
- [40] Hausknecht, M.; Stone, P. Deep Reinforcement Learning in Parameterized Action Space. In *Proceedings of the International Conference on Learning Representations (ICLR)*; 2016.
- [41] Xiong, J.; Wang, Q.; Yang, Z.; Sun, P.; Han, L.; Zheng, Y.; Fu, H.; Zhang, T.; Liu, J.; Liu, H. Parametrized Deep Q-Networks Learning: Reinforcement Learning with Discrete-Continuous Hybrid Action Space. *arXiv* **2018**, arXiv:1810.06394.